



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:

Sistemas Operativos

Asignación:

Tarea #2: Prioridad de procesos con Docker y límites de CPU

Profesor:

Prof. Isacio Tamayo

Estudiante:

James Tack

Grupo:

1IL143

Fecha de entrega:

18 de septiembre de 2026

2do Semestre 2026

De qué se trata la tarea

La tarea entrega un servicio web pequeño, app.py, hecho con Flask. Su ruta /login simula un inicio de sesión real, que primero espera dos segundos como si consultara una base de datos y después calcula hashes SHA-256 durante medio segundo de CPU. Hay que meterlo en un contenedor Docker con la CPU limitada y algunas variables de entorno, ponerlo a competir con 20 procesos que consumen CPU sin parar y ver qué le pasa al tiempo de respuesta. Al final hay que darle prioridad a app.py y demostrarlo con números.

Todo se hizo dentro de la máquina virtual del curso (vm-sistemas-operativos, Ubuntu 18.04 con 2 CPU y 2 GB de RAM), que quedó armada con Vagrant en el laboratorio 1. Docker 24.0.2 y Docker Compose 2.18.1 ya venían instalados por el aprovisionamiento, así que no hizo falta instalarlos de nuevo.

Cómo planteé el experimento

Al leer app.py noté que la ruta /start-cpu-stress lanza un script llamado stress.py que no venía con la tarea. Lo escribí yo. Crea tantos procesos como diga STRESS_WORKERS, y cada uno se queda en un ciclo while True que no hace otra cosa que gastar CPU. Usé procesos y no hilos. En Python los hilos comparten el GIL, así que 20 hilos de estrés se turnarían un solo candado y en la práctica se comportarían como uno, mientras que 20 procesos independientes sí compiten entre ellos y contra app.py en el planificador del kernel, que es lo que la tarea quiere observar.

Para decidir quién gana la CPU usé el valor nice de Linux, que va de −20 (más prioridad) a 19 (menos prioridad). Sin embargo, el planificador CFS no reparte la CPU por turnos iguales sino en proporción a un peso que sale del nice. Un proceso con nice 0 pesa 1024, con nice 10 pesa 110, con nice −10 pesa 9548 y con nice 19 apenas 15. Con esos pesos se puede calcular de antemano qué fracción de CPU le toca a app.py en cada caso, y después comparar con lo medido.

La configuración quedó en cuatro variables de entorno.

Variable	Valor	Qué controla
LOGIN_CPU_SECONDS	0.5	Segundos de CPU que gasta cada /login (app.py calibra las iteraciones al arrancar)
STRESS_WORKERS	20	Cantidad de procesos de estrés que lanza stress.py
APP_NICE	0, 10 o −10	Prioridad con la que arranca app.py (se aplica con nice -n en el CMD del Dockerfile)
STRESS_NICE	0 o 19	Prioridad que se pone cada proceso de estrés con os.setpriority

La delimitación de CPU va en docker-compose.yml, donde cpus "1.0" pone un tope de una CPU completa por medio de la cuota del cgroup, y cpuset "0" deja al contenedor solo en el núcleo 0. Así los 20 procesos y app.py pelean por el mismo núcleo, y en htop se ve un núcleo saturado y el otro libre. También agregué la capacidad SYS_NICE, porque Docker no la da por defecto y sin ella el kernel no deja bajar el nice por debajo de 0 dentro del contenedor.

Con eso armé cuatro escenarios.

Escenario	APP_NICE	STRESS_NICE	Estrés	Qué se quiere ver
E1	0	—	No	Tiempo base de /login sin competencia
E2	0	0	20 procesos	Competencia con la misma prioridad
E3	10	0	20 procesos	Los procesos de estrés tienen más prioridad que app.py
E4	−10	19	20 procesos	app.py priorizada sobre el estrés

Para medir escribí medir.py, que llama a /login varias veces seguidas, muestra los tiempos que devuelve app.py (espera, CPU y total) junto con el tiempo visto desde el cliente, y guarda cada petición en resultados.csv.

Preparación del entorno

Paso 1. Encender la máquina virtual

Levanté la VM con vagrant up desde la carpeta del laboratorio 1. El primer arranque se quedó detenido en la línea Loading essential drivers durante más de diez minutos. En el registro de VirtualBox aparece que corre sobre Hyper-V (NEM) porque AMD-V no está disponible en el anfitrión, y con ese motor el arranque a veces se cuelga. Un VBoxManage controlvm reset lo resolvió y la máquina terminó de arrancar en menos de un minuto.

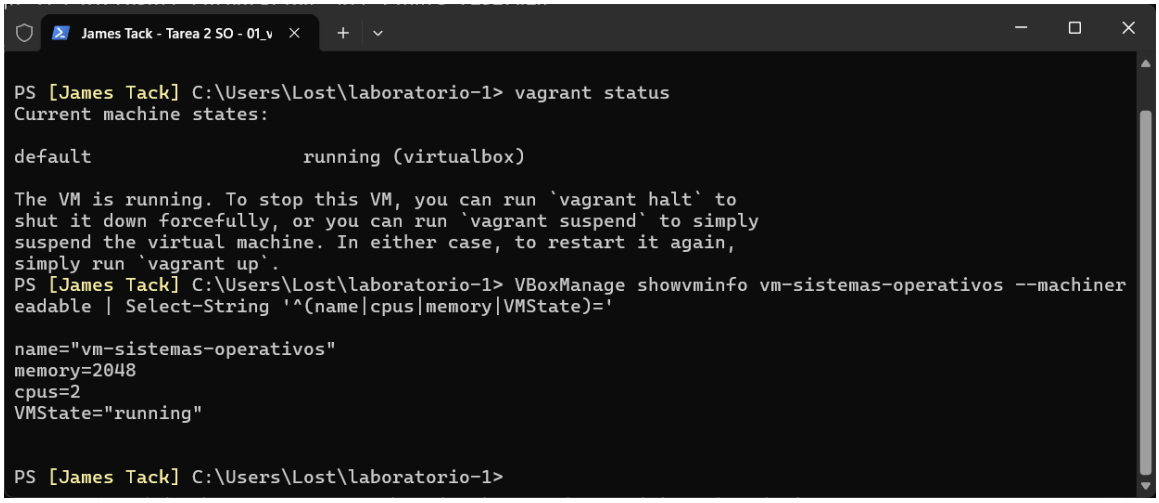


Figura 1. vagrant status y VBoxManage confirman que vm-sistemas-operativos está corriendo con 2 CPU y 2048 MB.

Paso 2. Copiar los archivos a la VM

Preparé en Windows la carpeta proyecto con app.py, stress.py, Dockerfile, docker-compose.yml y medir.py, y la copié a ~/laboratorios/tarea2 con scp usando la llave privada que genera Vagrant.

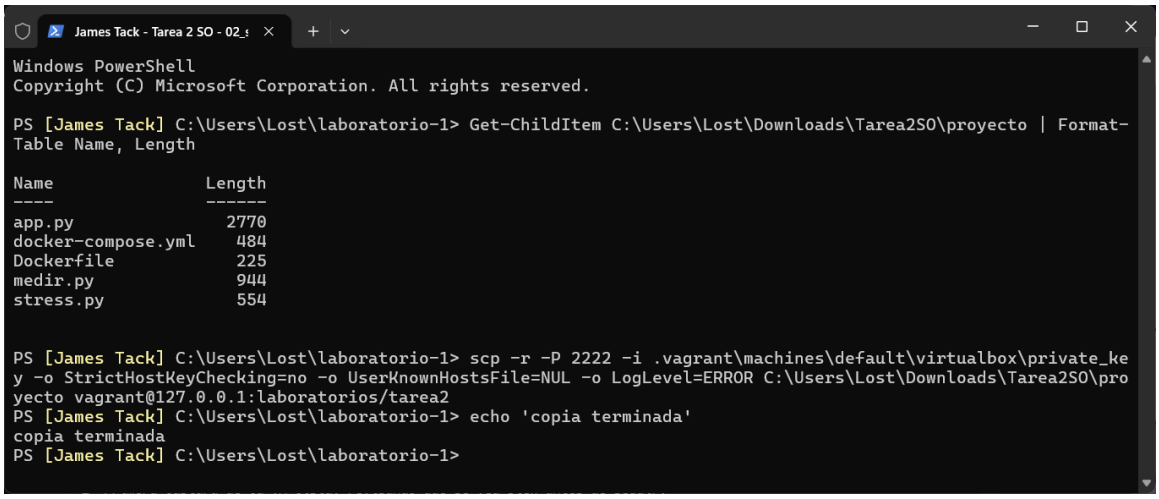


Figura 2. Archivos del proyecto en Windows y copia a la VM con scp.

Paso 3. Revisar el entorno dentro de la VM

Ya adentro revisé usuario, máquina, núcleos, memoria y versiones de Docker. Mi nombre sale en el prompt porque lo agregué al PS1 del .bashrc.

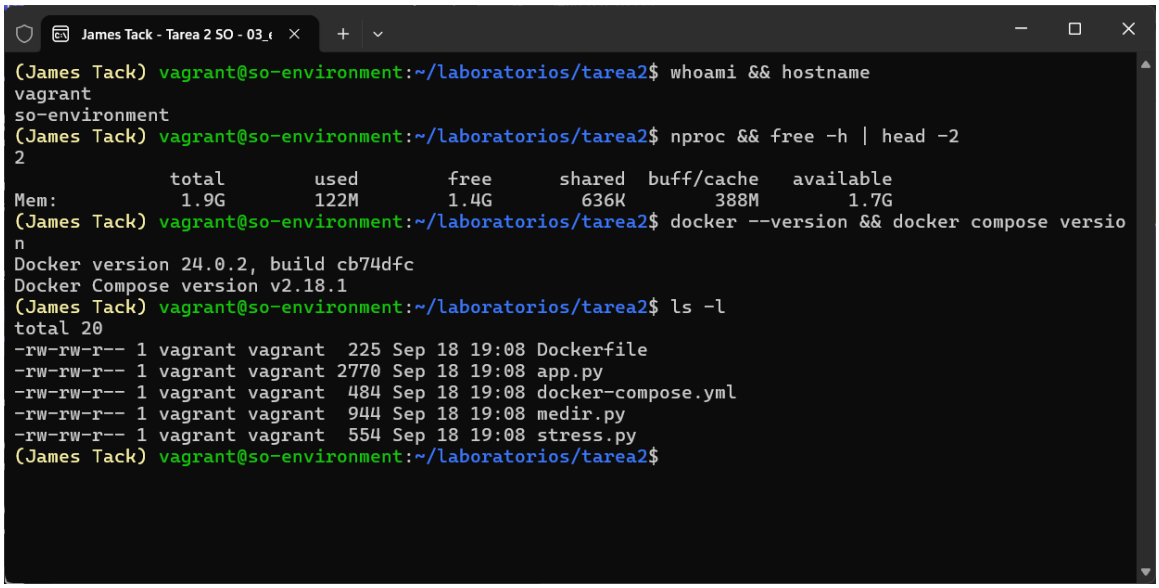


Figura 3. Dos CPU, 1.9 GB de RAM, Docker 24.0.2 y Compose 2.18.1 dentro de la VM.

Paso 4. Dockerfile, docker-compose.yml y stress.py

La imagen parte de python:3.11-slim y solo instala Flask. El CMD usa exec nice -n ${APP_NICE} python app.py, de modo que app.py queda como PID 1 del contenedor con la prioridad que diga la variable. En el compose están los límites de CPU, la capacidad SYS_NICE y las variables de entorno. APP_NICE y STRESS_NICE toman su valor de la shell al ejecutar docker compose up, y por eso cambiar de escenario no obliga a editar ningún archivo.

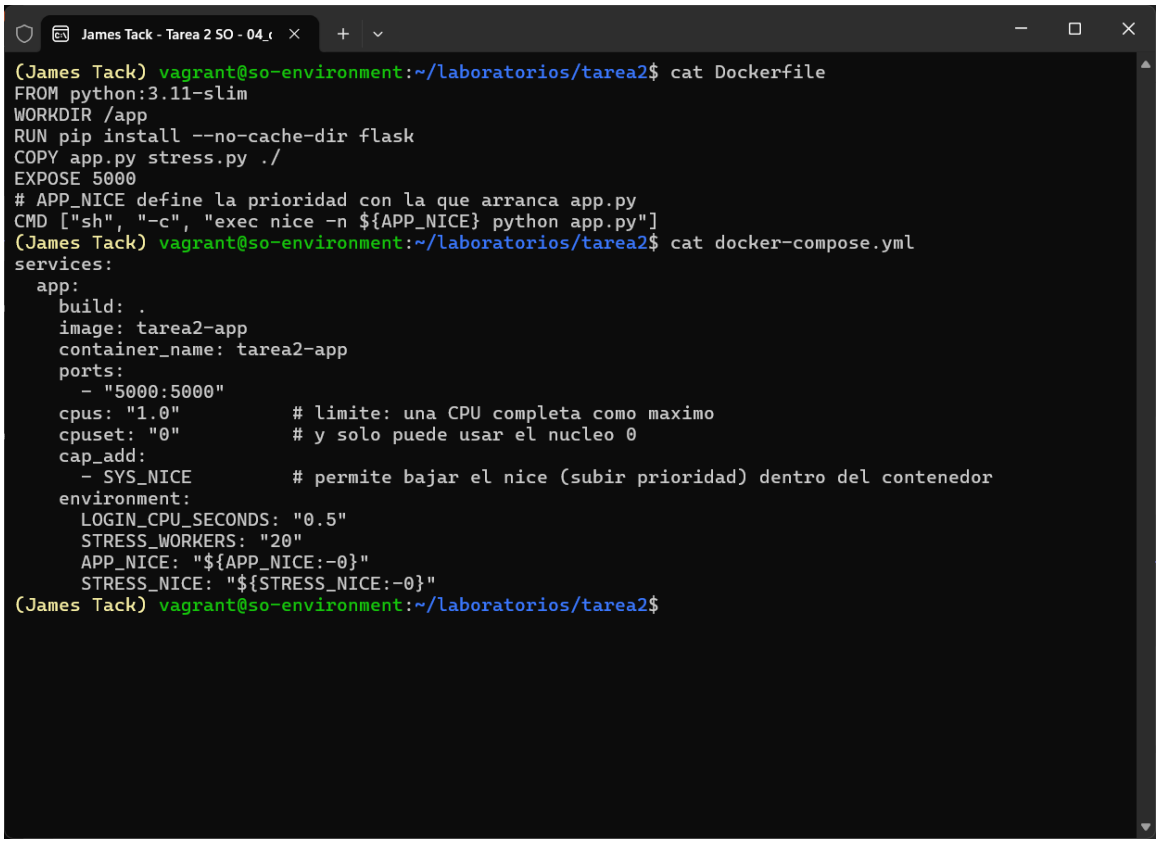


Figura 4. Dockerfile y docker-compose.yml con el límite de CPU, el cpuset y las variables de entorno.

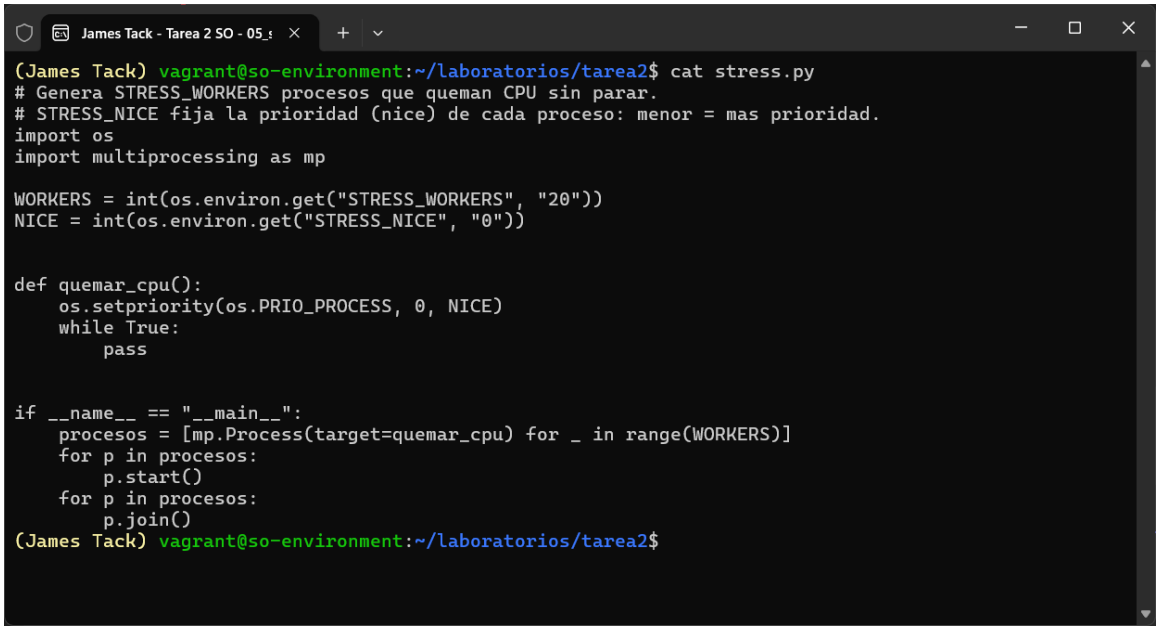


Figura 5. stress.py: 20 procesos que queman CPU con la prioridad indicada en STRESS_NICE.

Paso 5. Construir la imagen

```
docker compose build
```

Tardó 33 segundos y la imagen quedó de 149 MB.

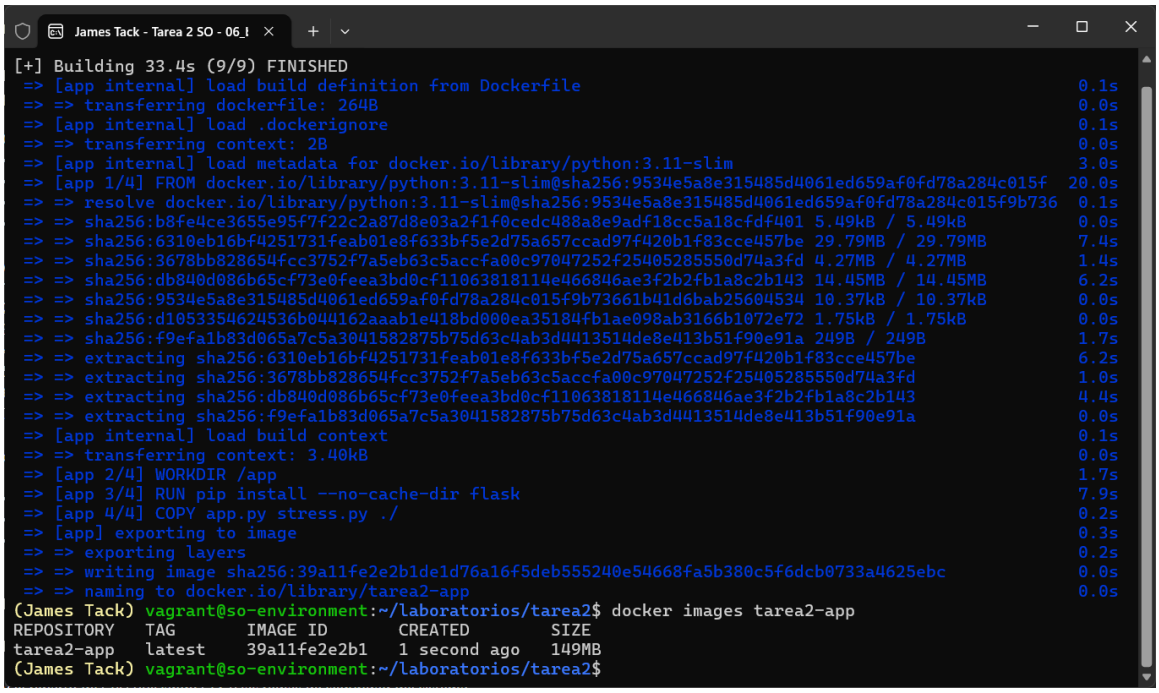


Figura 6. docker compose build termina y la imagen tarea2-app aparece en docker images.

Paso 6. Levantar el contenedor

Con docker compose up -d el contenedor arrancó con los valores por defecto (APP_NICE=0). La ruta /status confirma que app.py corre con nice 0, afinidad solo al núcleo 0 y 772711 iteraciones de hash calibradas para medio segundo.

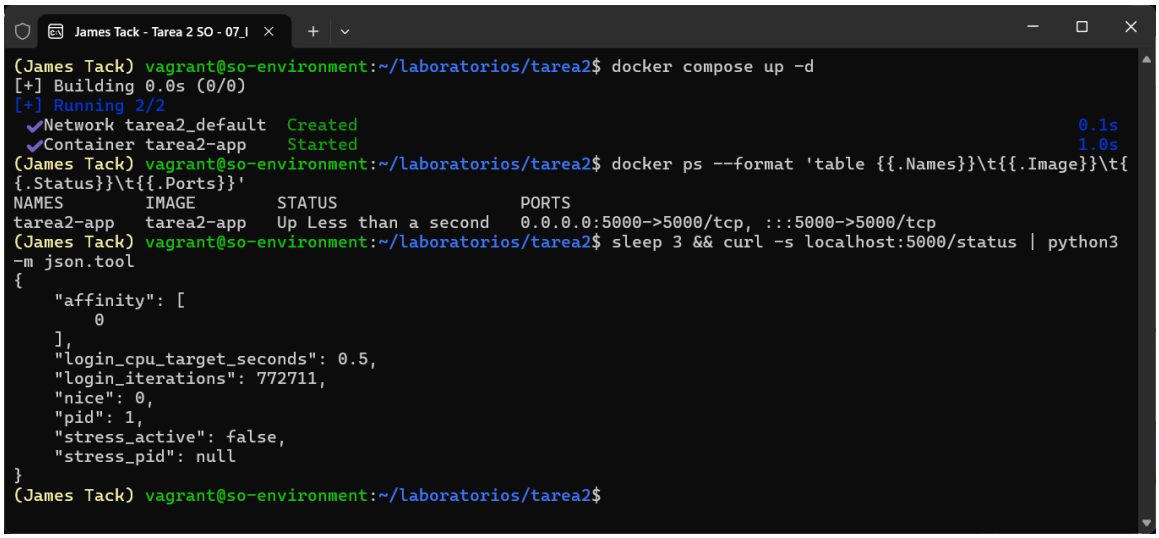


Figura 7. Contenedor tarea2-app en ejecución y respuesta de /status.

Escenarios y mediciones

E1. Sin estrés (línea base)

Antes de medir comprobé con docker inspect que los límites quedaron aplicados (NanoCpus=1000000000 equivale a una CPU, CpusetCpus=0 y CapAdd=[SYS_NICE]). Con docker exec revisé que las variables de entorno llegaron al contenedor.

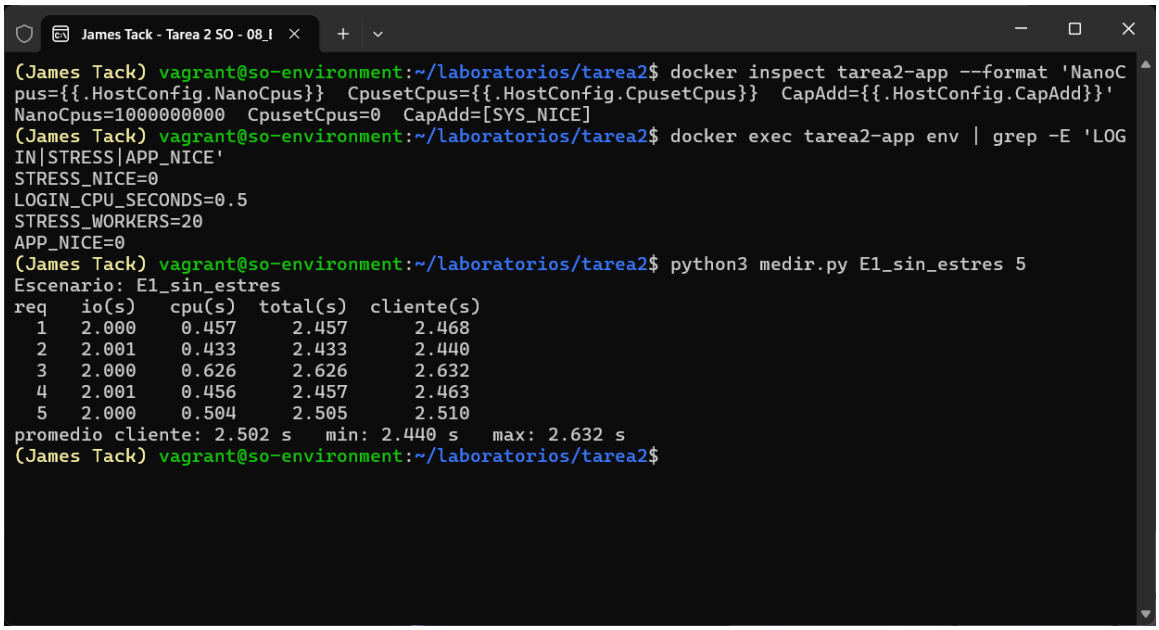


Figura 8. Límites del contenedor, variables de entorno y cinco peticiones a /login sin estrés.

Cada login tarda en promedio 2.502 s, que son los 2 s de espera más cerca de medio segundo de CPU. Contra ese número se compara todo lo demás.

E2. Estrés con la misma prioridad

Llamé a /start-cpu-stress y pgrep contó 21 procesos stress.py, el padre más los 20 que trabajan. docker stats marca el contenedor en 98.17 %, o sea, una CPU completa. El límite funciona, porque sin él 20 procesos ocupados se habrían comido los dos núcleos.

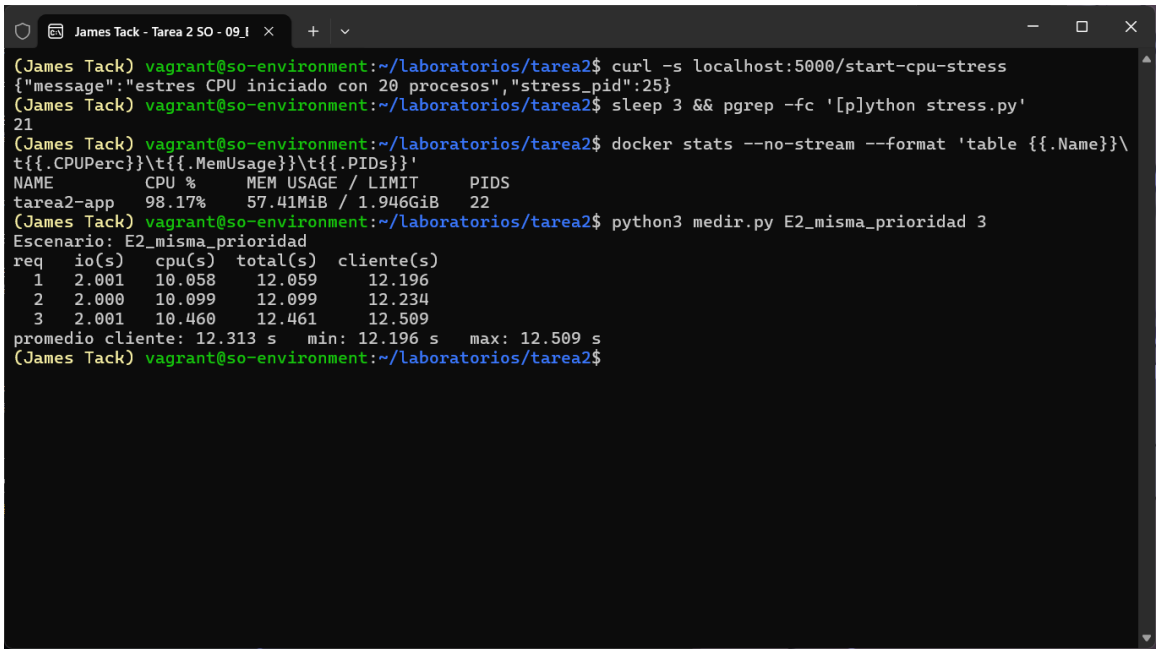


Figura 9. Inicio del estrés, 21 procesos, uso de CPU del contenedor y tiempos de /login con la misma prioridad.

El tiempo de login subió a 12.313 s. La espera de 2 s sigue igual porque dormir no usa CPU, y lo que se estira es la parte de cálculo, que pasó de 0.5 a 10.2 s. Con 21 procesos del mismo peso a app.py le toca 1/21 de la CPU, y 0.5 × 21 da 10.5 s, prácticamente lo medido.

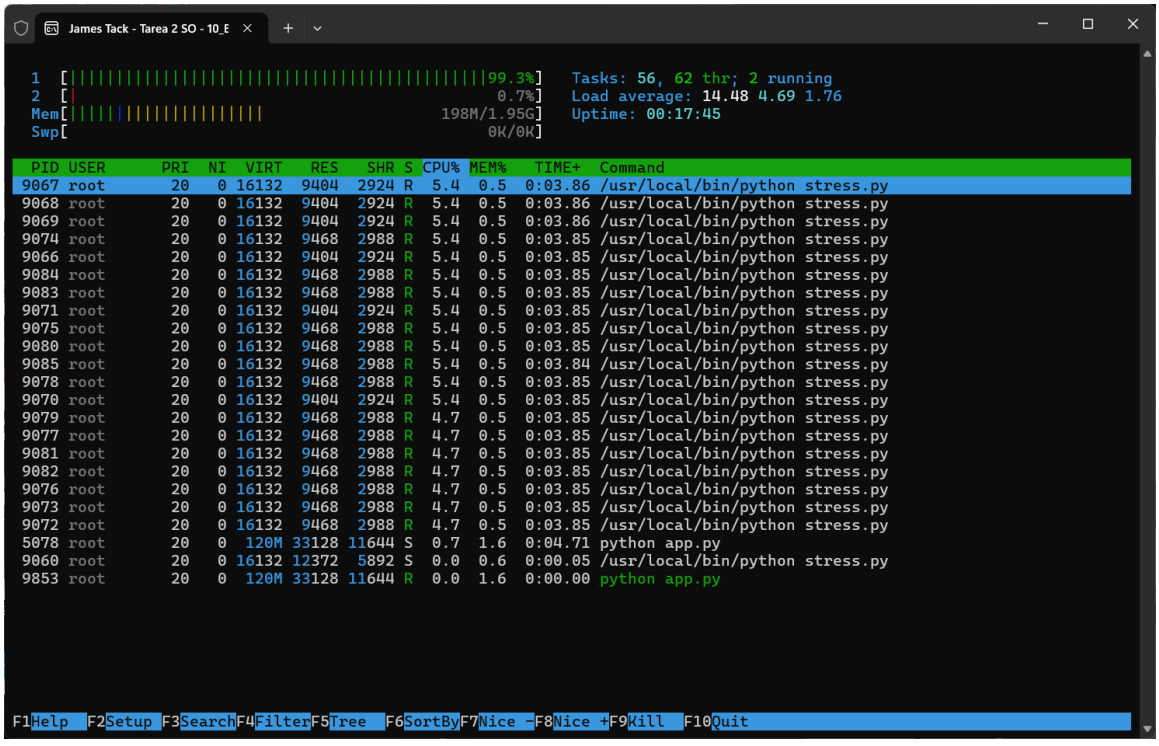


Figura 10. htop en E2: el núcleo 1 al 99.3 %, el núcleo 2 libre y los 20 procesos con NI 0 y cerca de 5 % de CPU cada uno.

En htop cada proceso de estrés recibe entre 4.7 y 5.4 % de la CPU, y app.py (PID 9853, en verde porque es un hilo) aparece en estado R esperando su turno como uno más.

E3. Estrés con más prioridad que app.py

Aquí app.py arranca con nice 10 y los 20 procesos de estrés quedan en nice 0, es decir, con más prioridad que el servicio. Por eso hay que recrear el contenedor con --force-recreate, ya que el nice de app.py se fija al arrancar y no cambia solo por modificar la variable. Es el caso que pide ver la tarea: un servicio compitiendo contra procesos más prioritarios.

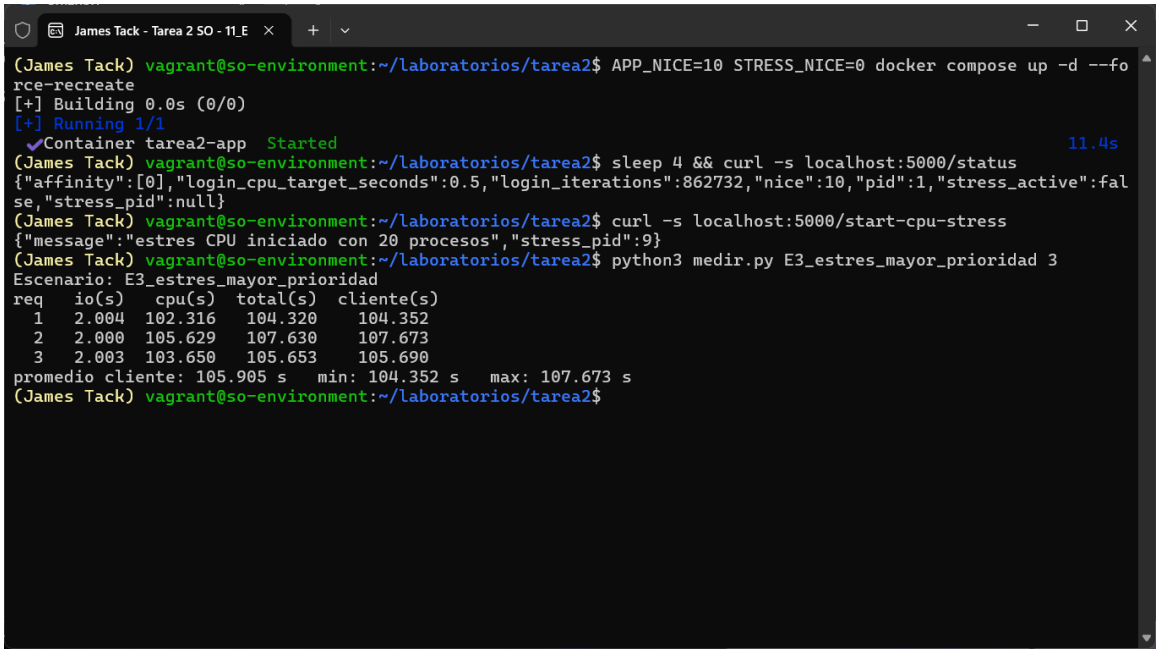


Figura 11. E3: app.py con nice 10 frente al estrés con nice 0; cada login tarda más de 100 s.

El login tardó 105.905 s. Son unas 42 veces el tiempo base. Según los pesos de CFS, app.py pesa 110 contra 20 × 1024 = 20480 del estrés, así que recibe cerca de 0.53 % de la CPU y el medio segundo de trabajo debería tardar unos 94 s. No obstante, se midieron 104 s de CPU, y la diferencia de unos 10 s probablemente viene del costo de cambiar de contexto entre 21 procesos y de que la cuenta supone que en ese núcleo no corre nada más.

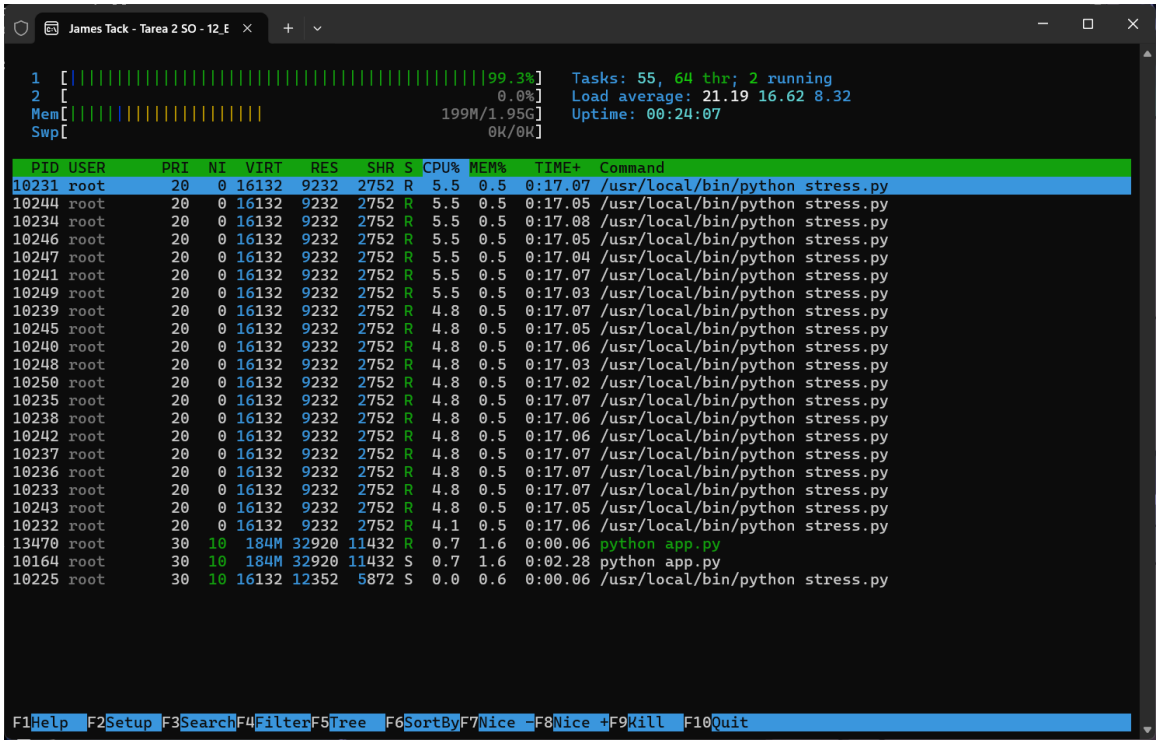


Figura 12. htop en E3: app.py con PRI 30 y NI 10 recibe 0.7 % de CPU mientras cada proceso de estrés recibe alrededor de 5 %.

En cambio, en htop la columna NI marca 10 para app.py y 0 para los procesos de estrés. El proceso padre de stress.py también sale con NI 10 porque lo hereda de app.py al ser lanzado, pero está dormido esperando a sus hijos y no compite.

E4. app.py con prioridad

Para darle prioridad a app.py la arranqué con nice −10 y bajé los procesos de estrés a nice 19. docker stats sigue marcando 100.86 %, el contenedor continúa usando su CPU completa y los 20 procesos siguen activos.

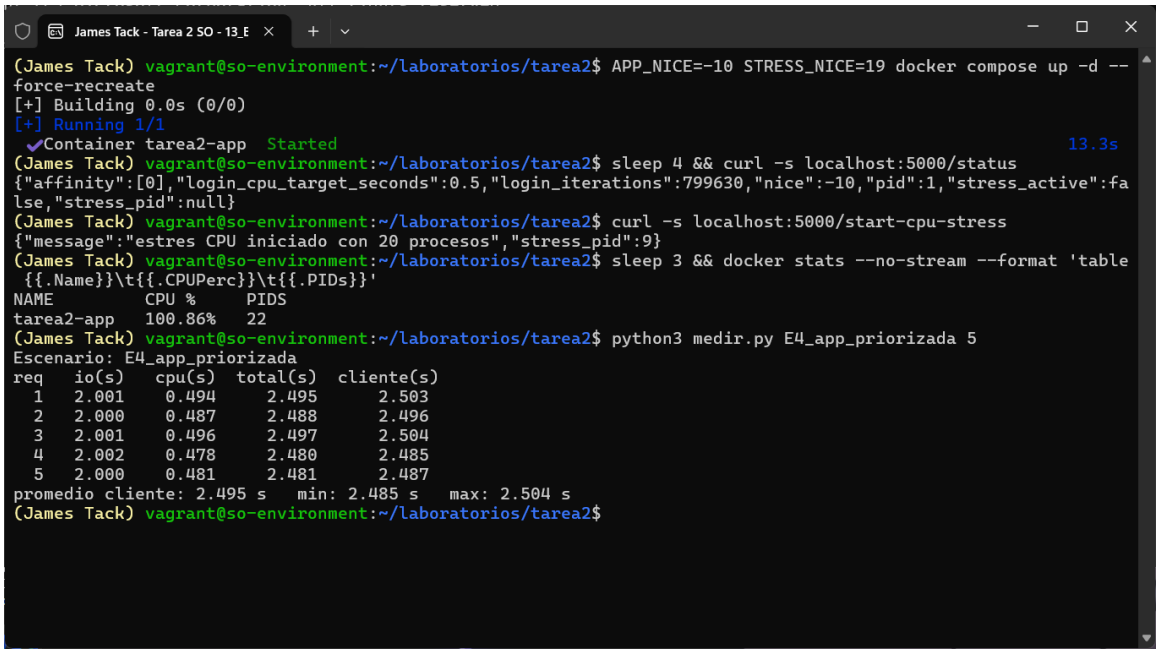


Figura 13. E4: app.py con nice −10 y el estrés con nice 19; el login vuelve a 2.5 s aunque el estrés sigue corriendo.

El login volvió a 2.495 s, igual que sin estrés. Con pesos de 9548 contra 20 × 15 = 300, app.py se queda con cerca de 97 % de la CPU cada vez que la necesita. El estrés no se detuvo. Solo aprovecha lo que app.py deja libre mientras duerme sus 2 s, y en cuanto el hilo despierta el planificador le devuelve la CPU casi entera porque su peso es más de treinta veces el de todos los procesos de estrés juntos.

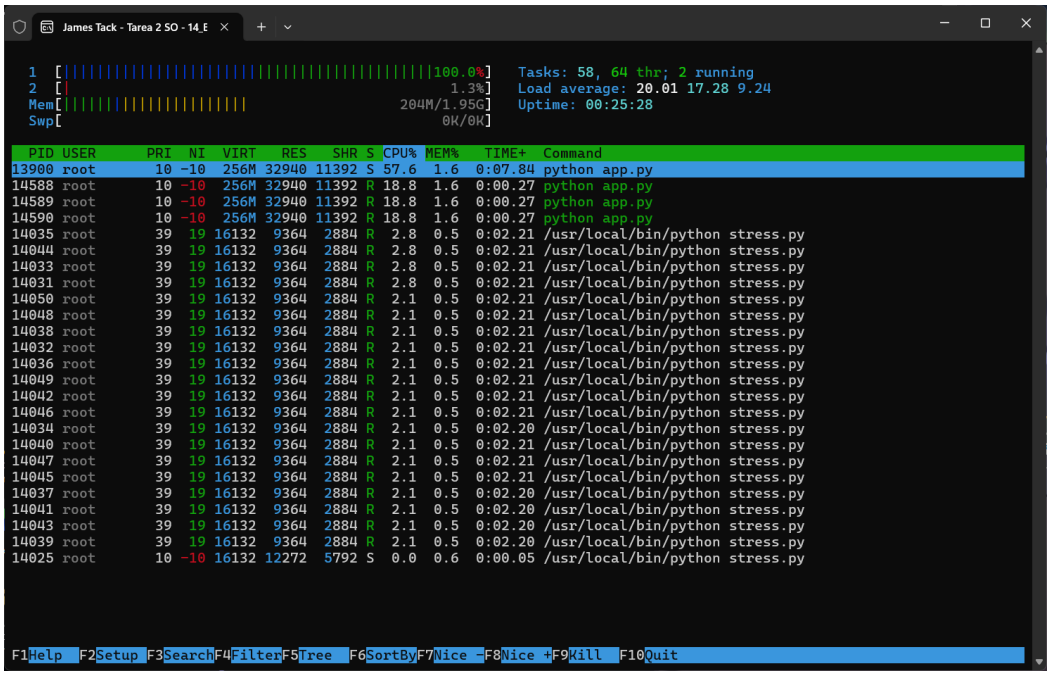


Figura 14. htop en E4: los hilos de app.py (PRI 10, NI −10) arriba de la lista y los procesos de estrés (PRI 39, NI 19) con 2 a 3 % cada uno.

Para esta captura lancé tres logins en paralelo, así se ven los tres hilos de Flask en estado R con 18.8 % cada uno y la línea del proceso app.py suma 57.6 %, arriba de los 20 procesos de estrés. La columna PRI pasa de 20 a 10 para app.py y de 20 a 39 para el estrés.

Resultados

Los tiempos de todas las peticiones quedaron en resultados.csv. El resumen por escenario lo saqué con awk dentro de la VM.

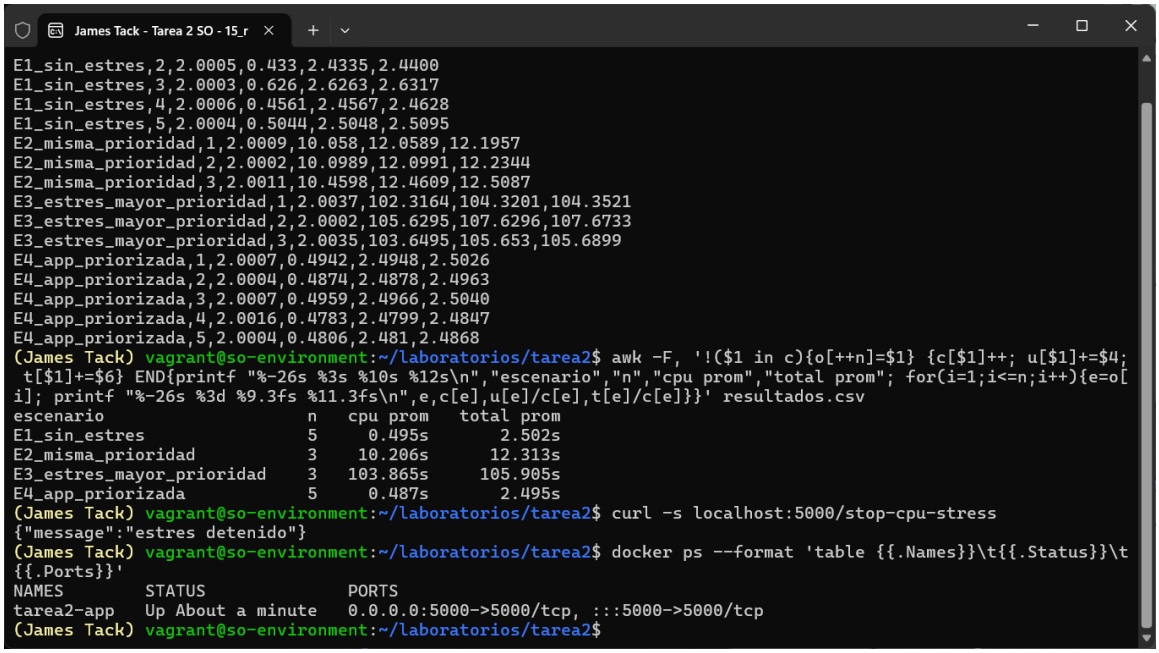


Figura 15. Contenido de resultados.csv y promedio por escenario.

Escenario	Peticiones	CPU prom. (s)	Total prom. (s)	Veces el base	CPU esperada según CFS
E1 sin estrés	5	0.495	2.502	1.0	100 %
E2 misma prioridad	3	10.206	12.313	4.9	1/21 ≈ 4.8 %
E3 estrés con más prioridad	3	103.865	105.905	42.3	110/20590 ≈ 0.53 %
E4 app.py priorizada	5	0.487	2.495	1.0	9548/9848 ≈ 97 %

La espera de 2 s no cambia en ningún escenario, porque mientras el hilo duerme no necesita CPU. Todo el efecto de la competencia cae sobre la parte de cálculo, que es donde entra el planificador. Entre E3 y E4 no cambió nada del contenedor salvo dos números de prioridad. Eran los mismos 20 procesos con el mismo límite y el mismo código, y aun así el login pasó de casi dos minutos a dos segundos y medio.

Lista de comprobación

Requisito de la tarea	Cómo se cumplió	Evidencia
Ambiente Docker para app.py	Dockerfile + docker-compose.yml, imagen tarea2-app	Figuras 4, 6 y 7
Delimitación de CPU	cpus: 1.0 y cpuset: 0	Figuras 4, 8 y 9
Variables de entorno	LOGIN_CPU_SECONDS, STRESS_WORKERS, APP_NICE, STRESS_NICE	Figura 8
Emular 20 procesos	stress.py con 20 procesos vía /start-cpu-stress	Figuras 9 y 10
Competencia con procesos de mayor prioridad	Escenario E3 (app nice 10, estrés nice 0)	Figuras 11 y 12
Dar prioridad a app.py	Escenario E4 (app nice −10, estrés nice 19)	Figuras 13 y 14
Demostración con tiempos y htop	medir.py, resultados.csv y capturas de htop	Figuras 10, 12, 14 y 15

Conclusiones

La prioridad de un proceso no se nota mientras sobra CPU. En E1 app.py tenía un núcleo para él solo y el nice daba igual. El problema aparece cuando hay más procesos listos que CPU disponible, que es justo lo que provoca el límite de una CPU con 20 procesos de estrés dentro.

Lo que más me sorprendió fue lo bien que se cumplen las cuentas del planificador. Con la tabla de pesos de CFS predije 10.5 s para E2 y salieron 10.2; para E3 predije unos 94 s y salieron 104. Linux no reparte la CPU a ojo. La reparte según un peso que se puede calcular de antemano, y el nice es solo la manera de elegir ese peso.

Además, el límite de Docker y la prioridad hacen cosas distintas. cpus y cpuset deciden cuánta CPU recibe el contenedor en total, mientras que el nice decide cómo se reparte esa CPU entre los procesos de adentro. En E4 el contenedor seguía al 100 % de su cuota y los 20 procesos seguían vivos, pero app.py respondía como si estuviera sola. Por otra parte, subir la prioridad dentro de un contenedor no sale gratis. Hizo falta agregar SYS_NICE, una capacidad que Docker quita por defecto para que un contenedor no pueda adueñarse de la CPU del anfitrión. En un servidor real conviene darla solo al servicio que la necesita, o dejar el reparto a los límites del propio Docker.

Anexos

Anexo A. Comandos ejecutados

Paso	Dónde	Comando	Para qué
1	Windows	vagrant up / vagrant status	Encender y comprobar la VM
1	Windows	VBoxManage controlvm vm-sistemas-operativos reset	Reiniciar la VM colgada en el arranque
2	Windows	scp -r -P 2222 -i <llave> proyecto vagrant@127.0.0.1:laboratorios/tarea2	Copiar el proyecto a la VM
3	VM	nproc / free -h / docker --version	Revisar recursos y Docker
5	VM	docker compose build	Construir la imagen tarea2-app

6	VM	docker compose up -d	Levantar el contenedor (E1)
E1	VM	docker inspect tarea2-app --format ...	Verificar NanoCpus, CpusetCpus y CapAdd
E1–E4	VM	python3 medir.py <escenario> <n>	Medir los tiempos de /login
E2–E4	VM	curl localhost:5000/start-cpu-stress	Lanzar los 20 procesos de estrés
E2–E4	VM	docker stats --no-stream	Ver el uso de CPU del contenedor
E2–E4	VM	htop -p $(pgrep -d, -f "[p]ython (app|stress).py")	Ver prioridad y CPU de cada proceso
E3	VM	APP_NICE=10 STRESS_NICE=0 docker compose up -d --force-recreate	Estrés con más prioridad
E4	VM	APP_NICE=-10 STRESS_NICE=19 docker compose up -d --force-recreate	app.py con prioridad
Final	VM	awk ... resultados.csv	Promedio por escenario

Anexo B. Archivos del proyecto

stress.py

```
# Genera STRESS_WORKERS procesos que queman CPU sin parar.
# STRESS_NICE fija la prioridad (nice) de cada proceso: menor = mas prioridad.
import os
import multiprocessing as mp

WORKERS = int(os.environ.get("STRESS_WORKERS", "20"))
NICE = int(os.environ.get("STRESS_NICE", "0"))


def quemar_cpu():
    os.setpriority(os.PRIO_PROCESS, 0, NICE)
    while True:
        pass


if __name__ == "__main__":
    procesos = [mp.Process(target=quemar_cpu) for _ in range(WORKERS)]
    for p in procesos:
        p.start()
    for p in procesos:
        p.join()
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN pip install --no-cache-dir flask
COPY app.py stress.py ./
EXPOSE 5000
# APP_NICE define la prioridad con la que arranca app.py
CMD ["sh", "-c", "exec nice -n ${APP_NICE} python app.py"]
```

docker-compose.yml

```
services:
  app:
    build: .
    image: tarea2-app
    container_name: tarea2-app
    ports:
      - "5000:5000"
    cpus: "1.0"          # limite: una CPU completa como maximo
    cpuset: "0"          # y solo puede usar el nucleo 0
    cap_add:
      - SYS_NICE         # permite bajar el nice (subir prioridad) dentro del contenedor
    environment:
      LOGIN_CPU_SECONDS: "0.5"
      STRESS_WORKERS: "20"
      APP_NICE: "${APP_NICE:-0}"
      STRESS_NICE: "${STRESS_NICE:-0}"
```

medir.py

```
#!/usr/bin/env python3
# Uso: ./medir.py <escenario> [peticiones]
# Llama a /login varias veces y guarda los tiempos en resultados.csv
import json, sys, time, urllib.request

escenario = sys.argv[1]
n = int(sys.argv[2]) if len(sys.argv) > 2 else 5
totales = []
print("Escenario:", escenario)
print("req   io(s)   cpu(s)  total(s)  cliente(s)")
for i in range(1, n + 1):
    t0 = time.time()
    d = json.load(urllib.request.urlopen("http://localhost:5000/login"))
    cliente = time.time() - t0
    totales.append(cliente)
    print("%3d  %6.3f  %7.3f  %8.3f  %9.3f" % (i, d["io_time_seconds"],
d["cpu_time_seconds"], d["total_time_seconds"], cliente))
    with open("resultados.csv", "a") as f:
        f.write("%s,%d,%s,%s,%s,%.4f\n" % (escenario, i, d["io_time_seconds"],
d["cpu_time_seconds"], d["total_time_seconds"], cliente))
print("promedio cliente: %.3f s   min: %.3f s   max: %.3f s" % (sum(totales) / n,
min(totales), max(totales)))
```